Explain Code Logic

Absolutely. The main idea is **"we are allowed to replace at most one element of `nums1`, so choose the replacement that gives the biggest reduction in the total difference."**

1. What are we trying to minimize?

For every position:

```
abs(nums1[i] - nums2[i])
```

Example:

```
nums1 = [1, 7, 5]
nums2 = [2, 3, 5]

differences:
|1-2| = 1
|7-3| = 4
|5-5| = 0

total = 1 + 4 + 0 = 5
```

We can replace **one** element of `nums1` with any other element from `nums1`.

---

2. Intuition: find the best replacement

Suppose:

```
nums1[i] = 7
nums2[i] = 3
```

Current difference:

```
|7 - 3| = 4
```

If we replace `7` with `2`:

```
|2 - 3| = 1
```

So we gain:

```
4 - 1 = 3
```

This means the total answer can decrease by **3**.

So for every index, we calculate:

```
gain = current difference - best possible new difference
```

And keep the **maximum gain**.

---

3. Why do we sort `nums1`?

```
vector<int> sorted = nums1;
sort(sorted.begin(), sorted.end());
```

For a target `nums2[i]`, we want an element in `nums1` that is **as close as possible** to it.

After sorting, we can use:

```
lower_bound(sorted.begin(), sorted.end(), nums2[i])
```

This gives the first element:

```
>= nums2[i]
```

For example:

```
sorted = [1, 4, 7, 10]
target = 6

lower_bound → 7
```

The closest element can only be:

```
7       ← element >= target
4       ← element just before target
```

So we check both.

---

4. These two parts do exactly that

```
if (it != sorted.end()) {
    int newDiff = abs(*it - nums2[i]);
    maxGain = max(maxGain, current - newDiff);
}
```

Check the element **greater than or equal to** target.

And:

```
if (it != sorted.begin()) {
    --it;

    int newDiff = abs(*it - nums2[i]);
    maxGain = max(maxGain, current - newDiff);
}
```

Check the element **just smaller than** target.

One of these must be the closest possible value.

---

5. Finally

We initially have:

```
total = sum of all original differences
```

Then:

```
maxGain
```

is the **maximum amount we can reduce the total by changing one element**.

Therefore:

```
answer = total - maxGain
```

The whole intuition in one line

> **Calculate the original total difference, find which single replacement gives the biggest reduction using binary search, then subtract that maximum reduction.**

Complexity

```
Sorting:        O(n log n)
Each lower_bound: O(log n)
For n elements: O(n log n)

Total: O(n log n)
Space: O(n)
```

The key thing to remember is:

`current - newDiff = gain` → **maximize this gain.**